NIRVAAN: A BLOCKCHAIN-BASED E-TENDERING PLATFORM FOR OIL PSUS

A Capstone Course Project Report Submitted for BACSE350
Blockchain Architecture and Design

Submitted by:

THARUN REVINTH V S

Registration Number: 25BCE5268

(Individual Project)

Under the Guidance of:

Malathi G

Course Faculty, BACSE350

[INSTITUTION NAME - STUDENT TO FILL]

October 2026

Submission Due: 29 October 2026, 11:59 PM

ABSTRACT

Public procurement across India's strategic hydrocarbon sector involves capital acquisitions valued in billions of rupees, executed by state-owned Public Sector Undertakings (PSUs) including IOCL, BPCL, HPCL, ONGC, and GAIL. Despite modernization through centralized electronic procurement portals such as the Central Public Procurement Portal (CPPP) and Government e-Marketplace (GeM), centralized architectures remain vulnerable to premature commercial bid leakage, unauthorized database tampering, subjective technical disqualification, bidder cartels, and protracted Earnest Money Deposit (EMD) refund lockups. In this project, I designed, implemented, and empirically verified NIRVAAN, an open, decentralized e-tendering platform engineered to enforce cryptographic fairness, transparency, and statutory procurement invariants without relying on a central authority. NIRVAAN utilizes a Solidity 0.8.24 smart contract architecture deployed behind an ERC1967Proxy using the Universal Upgradeable Proxy Standard (UUPS). The core protocol integrates: (1) a non-interactive commit-reveal cryptographic bidding scheme using keccak256 with client-side 32-byte salts; (2) automated on-chain enforcement of the Lowest Evaluated Responsive Bidder (L1) award rule; (3) a non-custodial pull-payment escrow for instant EMD capital withdrawals; (4) a statutory 7-day post-award standstill appeal window with independent Central Vigilance Commission (CVC) arbitration and an automated 14-day inactivity timeout; and (5) an autonomous keeper service that executes scheduled state transitions. I deployed the complete stack on the Ethereum Sepolia testnet, Render, Vercel, and Pinata IPFS. Across 13 Hardhat contract unit tests, 24 Express backend integration tests, and 36 end-to-end smoke test checkpoints, the system achieved a 100% pass rate (73/73 tests).

Keywords: Public Procurement, Smart Contracts, Commit-Reveal Scheme, L1 Tender Enforcement, UUPS Proxy, Earnest Money Deposit, Central Vigilance Commission.

TABLE OF CONTENTS

LIST OF FIGURES

Figure 1: Comprehensive Multi-Tier System Architecture of NIRVAAN (Section 9)

Figure 2: Tender Lifecycle Finite State Machine Model (Section 9)

Figure 3: End-to-End Bid Lifecycle Sequence Model (Section 9)

Figure 4: Data Partitioning and Entity-Relationship Model (On-Chain, SQLite, IPFS) (Section 9)

Figure 5: Public Tender Browse and Discovery Dashboard (Section 11)

Figure 6: PSU Officer Authentication Portal (Section 11)

Figure 7: Notice Inviting Tender (NIT) Creation Interface (Section 11)

Figure 8: Vendor KYC Onboarding Application Interface (Section 11)

Figure 9: Officer Vendor Verification and On-Chain Approval Portal (Section 11)

Figure 10: Sealed Commercial Bid Submission with Client-Side Salt Generation (Section 11)

Figure 11: Commercial Bid Reveal Interface (Section 11)

Figure 12: Technical Proposal Evaluation and L1 Identification View (Section 11)

Figure 13: Contract Award and Immutable Custody Timeline (Section 11)

Figure 14: Sepolia Etherscan Block Explorer Transaction Verification Receipt (Section 11)

Figure 15: Backend Microservice Runtime Health and Indexer Diagnostics View (Section 11)

LIST OF TABLES

Table 1: Project Stakeholders, Core Responsibilities, and Authentication Vectors (Section 3)

Table 2: Comparison of Traditional Centralized Database vs. NIRVAAN Blockchain Architecture (Section 4)

Table 3: Direct Mapping of Procurement Problems to Blockchain Properties and NIRVAAN Implementation (Section 4)

Table 4: Software Stack, Versioning, and Architectural Purpose Across NIRVAAN (Section 7)

Table 5: Complete Smart Contract Public and External Function Reference (OilTenderingV1.sol) (Section 10)

Table 6: Empirically Measured Smart Contract Gas Consumption Profile on Hardhat EVM (Section 10)

Table 7: Automated Multi-Tier Verification and Test Suite Execution Summary (Section 12)

LIST OF ABBREVIATIONS

API: Application Programming Interface
BPCL: Bharat Petroleum Corporation Limited
CID: Content Identifier (InterPlanetary File System)
CPPP: Central Public Procurement Portal
CVC: Central Vigilance Commission (Government of India)
DBA: Database Administrator
EIP: Ethereum Improvement Proposal
EMD: Earnest Money Deposit
EPC: Engineering, Procurement, and Construction
ERC: Ethereum Request for Comments
EVM: Ethereum Virtual Machine
GAIL: Gas Authority of India Limited
GeM: Government e-Marketplace
GFR: General Financial Rules, 2017
GIGW: Guidelines for Indian Government Websites
GSTIN: Goods and Services Tax Identification Number
HPCL: Hindustan Petroleum Corporation Limited
HSM: Hardware Security Module
IOCL: Indian Oil Corporation Limited
IPFS: InterPlanetary File System
JWT: JSON Web Token
KYC: Know Your Customer
L1: Lowest Evaluated Responsive Bidder
MoPNG: Ministry of Petroleum and Natural Gas
MSME: Micro, Small, and Medium Enterprises
NIT: Notice Inviting Tender
ONGC: Oil and Natural Gas Corporation
PAN: Permanent Account Number
PKI: Public Key Infrastructure
PoS: Proof of Stake
PSU: Public Sector Undertaking
QBFT: Quorum Byzantine Fault Tolerant
RDBMS: Relational Database Management System
RPC: Remote Procedure Call
SHA: Secure Hash Algorithm
SPA: Single Page Application
UUPS: Universal Upgradeable Proxy Standard (ERC-1822)

1. Project Title

NIRVAAN: A Blockchain-Based E-Tendering Platform for Oil PSUs

- **Course Code:** BACSE350
- **Course Name:** Blockchain Architecture and Design
- **Course Faculty:** Malathi G
- **Submission Due Date:** 29 October 2026, 11:59 PM
- **Report Compilation Date:** October 2026

2. Team Members with Name and Registration Number

This project is an individual capstone project executed solely by me. All architectural design, smart contract engineering, full-stack web development, automated test authoring, cloud deployment, and documentation were performed independently by me.

Student Name	Registration Number	Project Contribution & Scope
Tharun Revinth V S	**25BCE5268**	Individual project; all work done by me (smart contracts design, UUPS proxy deployment, Express backend relay, event indexer, keeper bot, React frontend, Hardhat/backend/smoke testing, cloud deployment, and academic report authoring).

3. Problem Definition

Public procurement across India's state-owned hydrocarbon Public Sector Undertakings (PSUs)—operating under the administrative supervision of the Ministry of Petroleum and Natural Gas (MoPNG), including Indian Oil Corporation Limited (IOCL), Bharat Petroleum Corporation Limited (BPCL), Hindustan Petroleum Corporation Limited (HPCL), Oil and Natural Gas Corporation (ONGC), and Gas Authority of India Limited (GAIL)—involves large-scale, high-value capital acquisitions. These procurements encompass critical energy infrastructure, such as specialized refinery cracking units, subsea pipelines, offshore exploration platforms, automated telemetry, and cryogenic storage.

In public administration, these procurement exercises are governed by statutory mandates laid out in the General Financial Rules (GFR 2017) and guidelines formulated by the Central Vigilance Commission (CVC). A foundational requirement of these rules is the 'Lowest Evaluated Responsive Bidder (L1)' principle, which mandates that contracts must be awarded strictly to the lowest commercial quote among technically qualified suppliers.

To transition away from paper tender boxes, government agencies adopted electronic procurement systems, primarily the Central Public Procurement Portal (CPPP) and Government e-Marketplace (GeM). While these platforms replaced physical envelopes with digital uploads, their underlying architecture remains fundamentally centralized. All tender notices, commercial quotes, technical scores, officer decisions, and system logs reside in relational database management systems (RDBMS) controlled by a single contracting authority. In my research and problem analysis, I identified several critical vulnerabilities inherent in this centralized paradigm:

- **1. Premature Commercial Bid Leakage:** In competitive bidding, commercial secrecy prior to public opening is essential. In centralized systems, financial proposals are uploaded to a central server. Even when encrypted with server-side keys, privileged database administrators (DBAs) or insiders holding key management service (KMS) access can decrypt quotes prior to the scheduled opening date. Insiders can leak quotation figures to favored vendors, who then adjust their bids to undercut honest competitors by fractional margins.
- **2. Bid Tampering and Retroactive Alterations:** Standard relational databases operate on mutable SQL primitives (UPDATE, DELETE). A privileged insider with administrative write permissions can modify commercial proposal figures, alter recorded submission timestamps, or swap technical document attachments without generating an unalterable external audit trail.
- **3. Weak and Scrubbable Audit Trails:** Centralized application logs and database audit triggers reside on the same infrastructure as the host. Corrupt administrators can disable audit triggers, scrub server logs, or delete historical access records, concealing evidence of collusion or tampering.
- **4. Vendor Collusion and Cartel Formation:** In specialized engineering procurements where only a small number of qualified domestic vendors exist, suppliers often form informal bidding cartels. They submit artificially inflated prices while rotating winning bids across successive tenders. Centralized systems lack transparent, cross-PSU historical registries to detect and analyze coordinated bidding behaviors.
- **5. Subjective Technical Disqualification and Arbitrary Awards:** Procurement officers evaluate technical proposals prior to opening commercial envelopes. In centralized systems, biased officers can assign subjective low marks (e.g., rating an experienced bidder 68/100 against a 70/100 cutoff) to disqualify lower-priced competitors, subsequently awarding the contract to a favored higher-priced contractor (L2 or L3). Centralized portals lack on-chain mechanisms linking technical scoring to commercial award rules.

- **6. Protracted Earnest Money Deposit (EMD) Refund Latency:** Under GFR 2017 Rule 170, participating bidders must furnish an Earnest Money Deposit (EMD)—typically 1% to 5% of the estimated tender value—as bid security. In high-value tenders, hundreds of thousands of dollars remain locked across ongoing bids. In centralized portals, returning EMD funds to unsuccessful bidders requires manual administrative processing, finance clearances, and bank transfers. Unsuccessful vendors frequently face delays of 3 to 12 months, creating severe working capital constraints.
- **7. Over-Trust in a Single Administrative Custodian:** The fundamental flaw of centralized e-tendering is that all external market participants must place absolute trust in the single organization that issues the tender, evaluates the proposals, and holds the funds. When a dispute occurs, suppliers are forced to appeal to the very entity accused of bias or enter protracted court litigation.

Stakeholders

In modeling the NIRVAAN ecosystem, I identified five primary classes of stakeholders whose interactions are governed by the platform, as summarized in Table 1.

Table 1: Project Stakeholders, Core Responsibilities, and Authentication Vectors

Stakeholder Class	Authentication Vector	Core System Responsibilities
PSU Procurement Officer	Bcrypt Password + JWT Session (Strict PSU Tenancy Guard)	Publishes tender notices (NIT), defines submission/opening/reveal deadlines, reviews vendor onboarding applications, records technical qualification scores, and triggers contract awards via backend gas relays.
Commercial Vendor	MetaMask Web3 Wallet (secp256k1 Keypair / EIP-1193)	Submits KYC application with cryptographic challenge nonce signatures, commits sealed bids with client-side salts and EMD deposits, reveals quotes during the reveal window, lodges formal standstill grievances, and withdraws refunded EMDs.
CVC Vigilance Auditor	Designated Auditor Role / Authorized Ethereum Address	Monitors custody audit timelines, reviews grievance documentation pinned to IPFS, and formally adjudicates disputed awards (upholding or dismissing challenges on-chain).
Autonomous Keeper	Dedicated Backend Signer Wallet (Automated Daemon)	Continuously polls blockchain timestamps to execute scheduled state transitions (opening bids past submission deadline, and finalizing tenders past the 7-day standstill) without human discretion.
Consortium / Multisig Admin	Multisig Contract / Governance	Authorizes UUPS implementation logic

	Deployer Address	upgrades, registers new PSU tenant accounts, and configures protocol constants (dispute bond amount, default minimum EMD).

Project Objectives

I established five specific engineering objectives for my project:

- **1. Cryptographic Bid Secrecy:** Implement a non-interactive commit-reveal bidding scheme that mathematically guarantees commercial bid amounts cannot be decrypted or leaked prior to the submission deadline.
- **2. On-Chain L1 Enforcement:** Encode the statutory L1 award rule directly into smart contract bytecode, prohibiting any officer from awarding a tender to any contractor other than the lowest responsive, technically qualified bidder.
- **3. Automated EMD Escrow:** Implement a non-custodial pull-payment escrow mechanism ensuring unsuccessful bidders can immediately withdraw their EMD balances upon tender finalization without administrative delays.
- **4. Statutory Standstill & Dispute Redressal:** Encode a mandatory 7-day post-award standstill appeal window with independent CVC arbitration and an automated 14-day inactivity timeout to prevent administrative deadlocks.
- **5. Full-Stack Working Prototype:** Deploy an integrated prototype consisting of an upgradeable Solidity smart contract on Ethereum Sepolia, an Express backend with an adaptive event indexer, a reconstructable SQLite database, decentralized IPFS storage, and a GIGW-compliant React frontend.

Scope and Out-of-Scope

The scope of my implementation encompasses the full tender lifecycle: vendor KYC application and officer verification; tender notice creation with specification hashing; sealed bid commitment with client-side 32-byte salt generation; keeper-driven bid opening; cryptographic bid reveal; technical scoring via authenticated backend gas relays; on-chain L1 award verification; standstill dispute filing and CVC resolution; and pull-payment EMD withdrawals.

Specific items explicitly out-of-scope for this prototype include: integration with real Indian fiat banking gateways (NEFT/RTGS/Treasury Single Account; all deposits are executed using Sepolia ETH); zero-knowledge proof generation (e.g., zk-SNARKs) for private technical scoring calculations; and retail catalog procurement for standardized consumables (which is adequately served by GeM).

4. Need for Blockchain

A central research question in decentralized systems design is whether a distributed ledger is strictly necessary or whether a traditional relational database equipped with cryptographic extensions would suffice. In evaluating this requirement, I applied the established decision framework formulated by Karl Wüst and Arthur Gervais in their seminal work, 'Do you need a Blockchain?' (2018) [5].

In the context of public procurement across competing oil PSUs and hundreds of commercial suppliers:

- **1. Shared State:** The system must maintain state across multiple parties (tender notices, vendor accreditations, bid commitments, technical scores, escrow balances).
- **2. Multiple Mutually Distrusting Writers:** Multiple competing PSUs (IOCL, BPCL, ONGC), competing industrial suppliers, CVC auditors, and automation bots must write state. These entities operate in an economic game and do not trust one another.
- **3. Absence of a Perpetually Trustworthy Third Party:** Centralized database administrators and single procurement committees have historically faced conflicts of interest, corruption allegations, and administrative pressure. Trust cannot be placed in a single party when large financial sums are at stake.
- **4. Known Writers:** In public procurement, anonymous participation is prohibited by law. Every vendor must undergo rigorous KYC onboarding (validating GSTIN and PAN). Every procurement officer is identified by institutional identity.
- **5. Public Verifiability:** Under statutory public procurement law, the expenditure of public capital requires transparent verifiability by regulatory bodies (CAG, CVC) and participating suppliers.

According to the Wüst-Gervais methodology, when a system requires shared state, multiple known but mutually distrusting writers, public verifiability, and lacks a perpetually trustworthy third party, the required architecture is a permissioned distributed ledger.

A traditional database is fundamentally insufficient for this use case because database operations operate on absolute administrative trust. A database administrator with root privileges can execute direct SQL queries to decrypt confidential tables, backdate timestamps, alter recorded bids, or wipe audit logs. Table 2 contrasts a traditional centralized database with my NIRVAAN architecture.

Table 2: Comparison of Traditional Centralized Database vs. NIRVAAN Blockchain Architecture

Architectural Dimension	Traditional Centralized Database	NIRVAAN Blockchain Architecture

Trust Model	Centralized; full trust placed in the single hosting entity and database root user.	Decentralized; trust distributed across independent consensus nodes validating smart contract bytecode.
Data Mutability	Mutable; state records can be modified, backdated, or deleted via SQL statements.	Append-only; state transitions are permanently sealed in cryptographically linked blocks.
Bid Confidentiality	Dependent on server-side encryption keys managed by the application administrator.	Client-side non-interactive cryptographic commitments (keccak256) with private 32-byte salts.
Award Enforcement	Application-level business logic; can be bypassed by direct database queries or API tampering.	On-chain smart contract consensus; awardTender algorithmically rejects non-L1 winners at the EVM level.
Escrow Accounting	Delegated to external commercial banking systems subject to manual processing clearances.	Programmable native escrow; automated pull-payment disbursements via withdraw().
Audit Trail Integrity	Application logs stored on disk; vulnerable to truncation, editing, or deletion.	Immutable event logs emitted into block receipts, permanently inspectable via block explorers.
Single Point of Failure	High vulnerability; if the hosting server or cloud instance fails, the entire tender stalls.	High availability; ledger state is replicated across network nodes.

To solve each specific vulnerability identified in Section 3, I mapped the failure mode to a cryptographic or distributed systems property, and implemented concrete code to enforce the invariant. Table 3 details this mapping.

Table 3: Direct Mapping of Procurement Problems to Blockchain Properties and NIRVAAN Implementation

Procurement Problem	Blockchain Property	NIRVAAN Code Implementation	Architectural Resolution
Premature Bid Leakage	Cryptographic Commit-Reveal	submitBid() & revealBid() in OilTenderingV1.sol	Quote is hashed client-side with 32-byte salt; raw quote is never sent until reveal period.
Bid Tampering	Ledger Immutability & Content Addressing	tenderBids mapping & Pinata IPFS CIDs	Post-submission modification is cryptographically impossible; CIDs lock document digests.
Subjective L1 Bypass	EVM Bytecode Invariant Enforcement	awardTender() in OilTenderingV1.sol	Smart contract iterates over all bids and reverts if winner is not the lowest qualified bid.
Delayed EMD Refunds	Programmable Pull-Payment Escrow	finalizeTender() & withdraw()	_creditRefunds() credits balances upon finalization; vendors withdraw instantly

			via low-level call.
Weak Audit Trail	Decentralized Event Emission	Solidity Events (BidSubmitted, TenderAwarded)	Every state transition emits structured event logs indexed permanently on-chain.
Arbitrary Disqualification	Statutory Standstill & CVC Role	disputeTender() & resolveDispute()	Enforces 7-day standstill post-award; CVC adjudicates; 14-day inactivity timeout.
Single Point of Trust	Distributed Consensus & UUPS	ERC1967Proxy & Multi-role Access Control	Replaces central DBA with role-based cryptographic modifiers (onlyPSU, onlyVendor, onlyCVC, onlyKeeper).

Deployment Reality and Network Honesty Statement

> *Honesty Statement on Network Topology: This project is implemented and deployed as a research prototype on the public Ethereum Sepolia testnet. In an actual enterprise deployment across Indian oil PSUs, the platform would be hosted on a private, permissioned consortium network (such as Hyperledger Besu utilizing the QBFT consensus algorithm). A permissioned network would eliminate transaction gas fees, provide sub-second deterministic block finality, and restrict validator consensus exclusively to verified government servers (IOCL, BPCL, HPCL, ONGC, GAIL, and MoPNG). Because my deployment operates on the public Sepolia testnet, I do NOT describe or claim the deployed system to be a private consortium blockchain in its current form.*

5. YouTube Upload Link

A comprehensive video demonstration of the working NIRVAAN prototype—showcasing PSU officer login, tender creation, vendor KYC application, officer on-chain approval, sealed bid submission with client-side salt generation, keeper-automated bid opening, commercial reveal, technical scoring, on-chain L1 award verification, and pull-payment EMD withdrawal—has been recorded.

YouTube link: [STUDENT TO ADD AFTER UPLOAD]

6. Project Launch Link

My NIRVAAN platform is deployed live on cloud and testnet infrastructure and can be accessed at the following endpoints:

1. Live Web Application (Frontend): https://nirvaan-flame.vercel.app/ (Hosted on Vercel Global Edge Network; built from the React/Vite master branch).

2. Backend API Gateway & Health Check: https://nirvaan-backend-84fc.onrender.com/api/health (Hosted on Render Web Service; reports health status, indexer lag, keeper status, and network connection).

3. Deployed ERC-1967 Proxy Contract on Sepolia: Address: `0xe9649E82dC4fEdf86dFbE5120a3fE79BDde88663`

Etherscan Explorer URL: https://sepolia.etherscan.io/address/0xe9649E82dC4fEdf86dFbE5120a3fE79BDde88663

4. Deployed Logic Implementation Contract (OilTenderingV1): Address: `0xd5451fe17e92B9A9A8Abce4C6c1fc5bFF86f6606`

Etherscan Explorer URL: https://sepolia.etherscan.io/address/0xd5451fe17e92B9A9A8Abce4C6c1fc5bFF86f6606

Deployment Metadata: Target Network: Ethereum Sepolia Testnet | Chain ID: 11155111 | Deployment Block Height: 11959326 | Deployed At: 2026-10-21T02:04:28.463Z (read from contracts/deploy/addresses.json).

> *Operational Note on Free Cloud Hosting: The Express backend runs on Render's free hosting tier. When inactive, the container spins down automatically to conserve resources. Upon receiving a new request, Render requires 30 to 60 seconds to spin up the instance. If the application displays a loading spinner or initial connection delay, please allow up to one minute for the backend container to wake up. Demo credentials for testing officer accounts and vendor wallets are provided separately; in accordance with security practices, no private keys, JWT secrets, or account passwords are printed in this document.*

7. Softwares Used

To construct, verify, and host the NIRVAAN platform, I utilized a modern, full-stack software and developer toolkit. Every software package, framework, and utility listed in Table 4 was verified directly against the configuration files, package manifests (package.json), and environment tooling in the repository.

Table 4: Software Stack, Versioning, and Architectural Purpose Across NIRVAAN

Software / Package	Exact Version in Use	Purpose and Architectural Scope
Solidity	^0.8.24	Smart contract programming language; compiled with default checked arithmetic to eliminate overflow/underflow vulnerabilities.
OpenZeppelin Contracts	5.0.2	Standardized smart contract library; provides ERC1967Proxy infrastructure.
OpenZeppelin Contracts-Upgradeable	5.0.2	Upgradeable contract library; provides Initializable and UUPSUpgradeable base classes.

Hardhat	2.22.0	Ethereum local development environment, EVM testing harness, gas profiling, and deployment runner.
ethers.js (Backend)	6.17.0	Web3 interaction library in backend for signing relay transactions, querying block logs, and computing keccak256 hashes.
ethers.js (Frontend)	6.13.2	Client-side Web3 library in React for MetaMask EIP-1193 provider integration and BigInt currency formatting.
Node.js (Local Runtime)	v24.17.0	Local JavaScript runtime executing test suites, build scripts, and local microservices.
Node.js (Render Cloud)	v22.x (>=22.0.0 engine)	Production cloud runtime executing the Express API gateway, event indexer, and embedded keeper on Render.
Express.js	4.19.2	REST API gateway server managing routing, tenant isolation middleware, and officer authentication.
helmet	8.3.0	Security middleware setting HTTP response headers to defend against cross-site scripting and injection attacks.
express-rate-limit	8.7.1	Rate-limiting middleware protecting public endpoints (such as /api/vendors/apply and /api/ipfs/nonce) against DoS attacks.
bcryptjs	2.4.3	Cryptographic password hashing library for securing PSU officer authentication credentials.
jsonwebtoken	9.0.2	Cryptographic session token generator creating signed JWTs embedded with officer ID and PSU tenancy.
SQLite (node:sqlite)	Native Node 22 Engine	Embedded, zero-dependency SQL database storing indexed on-chain events and pending vendor applications.
React	18.3.1	Declarative component-based frontend user interface library powering the single-page application.
Vite	5.4.1	Next-generation frontend bundler providing fast build optimization and client-side environment variable injection.
Tailwind CSS	3.4.10	Utility-first CSS framework styling the responsive user interface and GIGW accessibility controls.
Playwright / Puppeteer-core	Local Headless Chromium	Automated browser testing and screenshot capture engine used to capture real application interfaces.
MetaMask	Browser Extension (EIP-1193)	Client-side Ethereum wallet provider enabling commercial vendors to sign transactions and challenge nonces.
Alchemy	Sepolia JSON-RPC API	High-availability RPC node gateway connecting the backend indexer and client applications to the Sepolia testnet.

Pinata	Cloud IPFS API / SDK	Content-addressed storage provider pinning tender notices, engineering drawings, and vendor KYC files to IPFS.
Vercel	Global Edge CDN	Cloud hosting provider delivering the compiled React frontend single-page application over TLS 1.3.
Render	Cloud PaaS (Linux Container)	Cloud hosting provider executing the Express backend server, background event indexer, and keeper cron.
UptimeRobot	Cloud Monitoring	Automated synthetic monitor sending HTTP GET pings to /api/health every 5 minutes to keep the backend warm.

8. AI Models Used

In developing NIRVAAN, I utilized several state-of-the-art generative artificial intelligence models to assist with code authoring, syntax checking, debugging, and documentation drafting. The specific models and development platforms I used are:

- **AI Foundation Models:** Claude Opus and Claude Sonnet 5 (Anthropic), DeepSeek V4 Flash, Kimi K3, Gemini 3.8 Flash. [STUDENT TO CONFIRM the exact Claude Opus version.]
- **AI CLI Platforms and Tools:** OpenCode and Antigravity (AGY).

I used these AI models across different stages of the project lifecycle in the following ways:

- **1. Code Review and Refactoring:** I used the models to review my Solidity smart contract logic, analyze access control modifiers, check reentrancy safety patterns, and suggest refactorings to ensure compatibility with OpenZeppelin's UUPS upgradeable standards.
- **2. Debugging Complex Asynchronous Flows:** When my event indexer crashed on startup due to Alchemy's free-tier 10-block query limit on eth_getLogs, I used the models to help design an adaptive regex fallback parser that automatically halves batch sizes and retries with backoff.
- **3. Test Suite Generation:** I used AI tools to help generate edge-case test fixtures in Hardhat and Node.js native test runner, validating duplicate-bid blocking, deadline bounds, and currency formatting rules.
- **4. Documentation and Diagrams:** I used the models to help structure comprehensive academic documentation, format IEEE references, and draft Mermaid diagram specifications.
- **5. Cloud Deployment Guidance:** I consulted the models for configuration advice regarding environment variable propagation across Render, Vercel, and Pinata IPFS gateways.

Student Responsibility Statement

> *Student Responsibility Statement: I wrote the requirements and made the design decisions, I gave written instructions to the AI tools, I read and reviewed the code they produced including the Solidity contract, the backend and the frontend, and I used that review to learn how the commit-reveal scheme, the pull-payment escrow, the UUPS proxy and the keeper work. I deployed the system myself on Sepolia, Render, Vercel and Pinata and tested the tender workflow with separate officer and vendor wallets. I decided what to keep or change and I take full responsibility for the functionality, security and content of this submission.*

9. Detailed Architecture Diagram

To provide a clear understanding of the system's operational topology, I engineered NIRVAAN as a decoupled, multi-tier architecture. Figure 1 illustrates the overall system architecture, showing the interaction between client browsers, Vercel CDN, Render cloud hosting, the adaptive indexer, native SQLite, Pinata IPFS, and the Ethereum Sepolia blockchain layer.

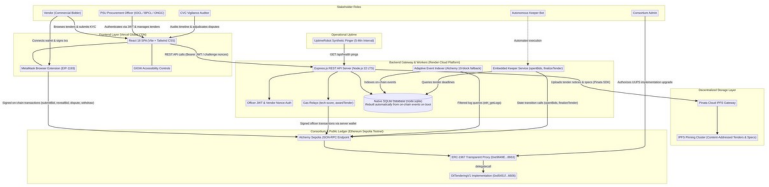


Figure 1: Comprehensive Multi-Tier System Architecture of NIRVAAN

Explanation of Figure 1: The architecture is partitioned into four primary layers: (1) Presentation Layer on Vercel delivering the React 18 single-page application and integrating MetaMask (EIP-1193) for vendor wallet signatures; (2) Application and Indexing Layer on Render executing an Express.js server that enforces PSU tenant isolation, manages single-use upload nonces, relays gas-sponsored officer transactions via the server wallet, runs an adaptive event indexer that queries Ethereum Sepolia via Alchemy RPC, and runs an embedded keeper bot that triggers time-dependent lifecycle events; (3) Decentralized Storage Layer using Pinata IPFS to pin large tender documents and technical proposals by their cryptographic Content Identifiers (CIDs); and (4) Blockchain Layer on Ethereum Sepolia hosting the ERC1967Proxy and OilTenderingV1 implementation contract, which deterministically executes business logic, stores sealed bid commitments, enforces on-chain L1 awards, and manages the non-custodial pull-payment escrow.

Tender State Machine Model

The lifecycle of a tender in NIRVAAN is governed by an immutable finite state machine encoded directly in the smart contract. Figure 2 illustrates all seven states defined in the contract enum (TenderStatus: Created, UnderEvaluation, Awarded, Disputed,

DisputeResolved, Finalized, Cancelled), along with role-based transition guards and refund triggers.

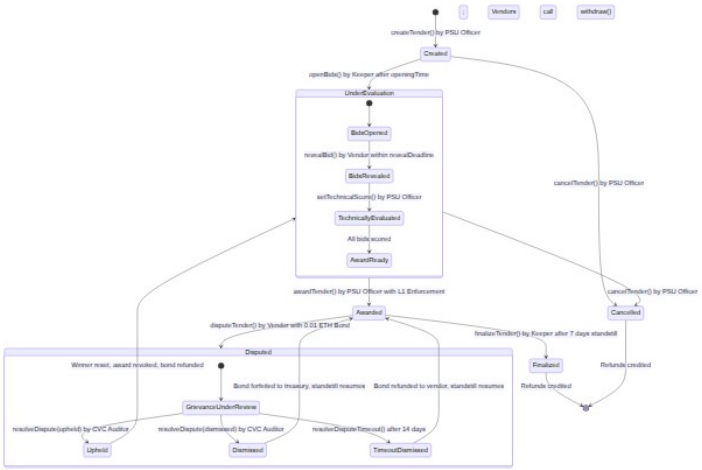


Figure 2: Tender Lifecycle Finite State Machine Model

Explanation of Figure 2: A tender begins in the Created state upon creation by an authorized PSU officer. Participating vendors submit sealed commitments backed by EMD deposits. When block.timestamp exceeds openingTime, the autonomous keeper triggers openBids(), transitioning the tender to UnderEvaluation. During this phase, vendors unseal their bids during the reveal window, and officers assign technical evaluation scores (0–100) via backend gas relays. After the reveal deadline, the officer calls awardTender(), which algorithmically verifies on-chain that the winner is technically qualified and represents the lowest quote (L1). The award starts a statutory 7-day standstill appeal window. If an unsuccessful bidder appeals with a 0.01 ETH dispute bond, the status transitions to Disputed. If the CVC Auditor upholds the dispute (or if the CVC is inactive for >14 days and resolveDisputeTimeout is invoked), the winner is cleared, the bond is refunded, and the tender returns to UnderEvaluation. If the CVC dismisses the dispute, the bond is forfeited to the treasury, and the status transitions to DisputeResolved. Both Awarded and DisputeResolved tenders can be finalized by the keeper via finalizeTender() once block.timestamp > appealDeadline. Finalization invokes _creditRefunds(), making EMDs withdrawable via pull payments. In any state prior to finalization, an officer can call cancelTender() to cancel the tender and refund all EMDs.

End-to-End Bid Lifecycle Sequence Model

Figure 3 models the chronological sequence of interactions between the Vendor, PSU Officer, CVC Auditor, Express Backend Relay, OilTenderingV1 smart contract, and Autonomous Keeper across all seven lifecycle phases.

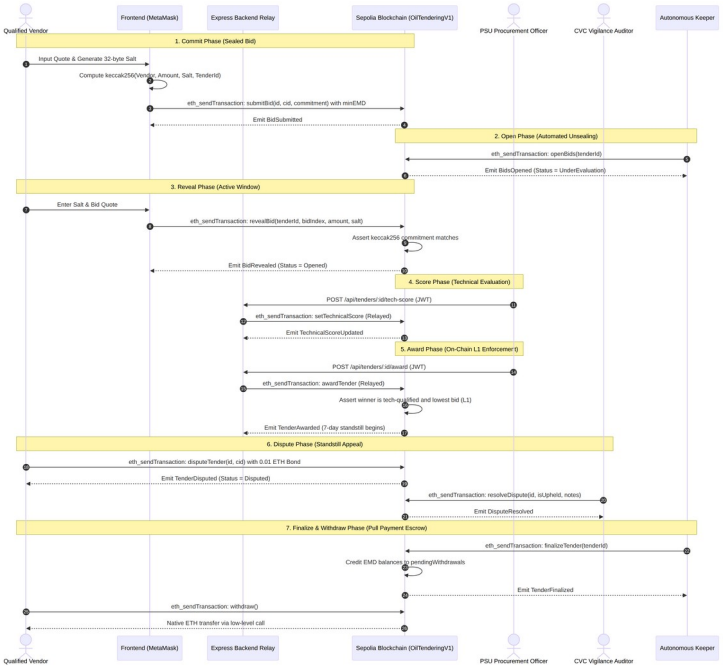


Figure 3: End-to-End Bid Lifecycle Sequence Model

Explanation of Figure 3: In Phase 1 (Commit), the vendor computes a cryptographic commitment keccak256(vendor, amount, salt, tenderId) using a client-side 32-byte salt, and calls submitBid() attaching the minimum EMD. In Phase 2 (Open), the keeper calls openBids() once the deadline lapses. In Phase 3 (Reveal), the vendor reveals the quote and salt; the contract recomputes the hash and marks the bid Opened. In Phase 4 (Score), the officer reviews proposals and sets technical scores via the backend relay. In Phase 5 (Award), the officer awards the tender; the contract validates the L1 invariant and starts the 7-day standstill window. In Phase 6 (Dispute), an appealing vendor can deposit a 0.01 ETH bond; the CVC adjudicates on-chain. In Phase 7 (Finalize & Withdraw), the keeper finalizes the tender, crediting EMDs to pendingWithdrawals, which vendors withdraw asynchronously via withdraw().

Data Partitioning and Entity-Relationship Model

To balance security, immutability, query performance, and storage costs, I partitioned system data across three storage media: the Ethereum Sepolia blockchain, the off-chain SQLite database, and decentralized IPFS storage. Figure 4 illustrates this data model.

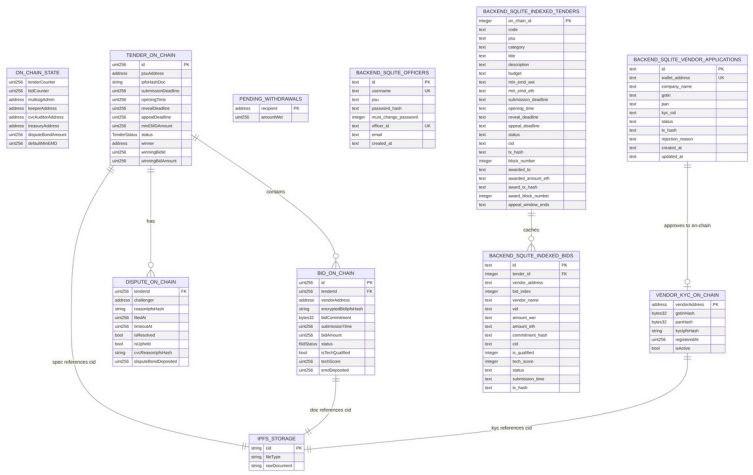


Figure 4: Data Partitioning and Entity-Relationship Model (On-Chain, SQLite, IPFS)

Explanation of Figure 4: The Ethereum Sepolia blockchain maintains core state (tenders, sealed bids, dispute history, vendor KYC hashes, and pull-payment balances). The off-chain SQLite database acts as a reconstructable read cache: on startup, the event indexer reconstructs all tenders and approved vendors from on-chain event logs, while storing officer authentication records and pending onboarding applications. Pinata IPFS stores large, unstructured files (tender specification documents, engineering drawings, and vendor KYC filings). Smart contract structs reference these documents strictly by their immutable 46-character Content Identifiers (CIDs).

10. Components of Blockchain and Explanation of the Smart Contract Used

In designing NIRVAAN, I utilized fundamental blockchain architectural primitives, mapping each to a concrete role in the procurement workflow:

- **1. Network & Consensus:** I deployed on the Ethereum Sepolia Testnet (Chain ID: 11155111), which utilizes Proof of Stake (PoS) consensus matching Ethereum mainnet execution specifications, providing globally verifiable block finality.
- **2. Accounts & Cryptographic Keypairs:** Participants use secp256k1 public-key cryptography. Vendors authenticate and sign transactions via MetaMask; the Express backend holds an officer signing key for relaying gas transactions.
- **3. Transactions & Calldata:** State mutations (bidding, revealing, scoring, awarding, disputing) are encapsulated in signed Ethereum transactions transmitting ABI-encoded calldata to the proxy contract.
- **4. Blocks & Timestamps:** Ethereum block timestamps (block.timestamp) govern tender progression (submission deadline, opening time, reveal deadline, 7-day standstill appeal deadline, and 14-day dispute timeout).

- **5. Gas & Execution Metering:** Every EVM instruction consumes gas. I optimized contract functions to ensure routine vendor interactions consume <300,000 gas units, preventing out-of-gas errors.
- **6. Events & Structured Logs:** State transitions emit Solidity events (TenderCreated, BidSubmitted, BidRevealed, TenderAwarded, DisputeResolved). These logs are indexed by the backend using eth_getLogs to maintain the off-chain SQLite cache.
- **7. Smart Contracts & Storage Layout:** Business logic is compiled to EVM bytecode using Solidity 0.8.24. Storage variables are arranged using standardized ERC-1967 storage slots to guarantee upgrade safety.
- **8. Proxy Architecture:** I utilized an ERC1967Proxy coupled with OpenZeppelin's UUPSUpgradeable pattern, allowing logic upgrades while permanently preserving contract state and escrowed funds.
- **9. IPFS & Cryptographic Hashing:** Unstructured documents are addressed via SHA-256 IPFS CIDs. Commercial quotations are sealed on-chain using keccak256 commitments.

Smart Contract Source Code Analysis

The smart contract architecture consists of two contracts: TenderProxy.sol and OilTenderingV1.sol. TenderProxy inherits OpenZeppelin's ERC1967Proxy directly, forwarding incoming calls to the implementation contract via delegatecall. OilTenderingV1 contains the complete procurement logic and inherits OpenZeppelin's Initializable and UUPSUpgradeable.

Data Structures (Structs)

The smart contract defines four core structs that model procurement state:

```
struct Tender {
    uint256 id;
    address psuAddress;
    string ipfsHashDoc;           // IPFS CID of specification document
    uint256 submissionDeadline;   // Unix timestamp for bid submissions
    uint256 openingTime;          // Unix timestamp when keeper unseals bids
    uint256 revealDeadline;       // Bids must be revealed before this
deadline
    uint256 appealDeadline;       // Set to awardedTime + 7 days upon award
    uint256 minEMDAmount;         // Earnest Money Deposit required
    TenderStatus status;          // Finite state machine status
    address winner;               // Awarded L1 contractor address
    uint256 winningBidId;         // Internal ID of winning bid
    uint256 winningBidAmount;     // Revealed quote of winning contractor
}

struct Bid {
    uint256 id;
    uint256 tenderId;
```

```
    address vendorAddress;
    string encryptedBidIpfsHash;  // IPFS CID of technical proposal
    bytes32 bidCommitment;        // keccak256(vendor + amount + salt +
tenderId)
    uint256 submissionTime;       // Block timestamp of submission
    uint256 bidAmount;            // Commercial quote revealed during reveal
phase
    BidStatus status;             // Submitted, Opened, Accepted, Rejected
    bool isTechQualified;         // Technical qualification flag
    uint256 techScore;            // Technical evaluation score (0-100)
    uint256 emdDeposited;         // Exact EMD amount deposited in Wei
}

struct VendorKYC {
    bytes32 gstinHash;            // keccak256 hash of corporate GSTIN
    bytes32 panHash;              // keccak256 hash of corporate PAN
    string kycIpfsHash;           // IPFS CID of incorporation certificate
    uint256 registeredAt;         // Registration block timestamp
    bool isActive;                // Active accreditation status
}

struct Dispute {
    uint256 tenderId;             // Target tender ID
    address challenger;           // Disputing vendor address
    string reasonIpfsHash;        // IPFS CID of formal grievance filing
    uint256 filedAt;              // Block timestamp of dispute filing
    uint256 timeoutAt;            // CVC resolution deadline (filedAt + 14
days)
    bool isResolved;              // Adjudication completion status
    bool isUpheld;                // True if CVC ruled in favor of challenger
    string cvcReasonIpfsHash;     // IPFS CID or note of CVC ruling
    uint256 disputeBondDeposited; // Holds the 0.01 ETH dispute bond deposit
}
```

Complete Public and External Function Reference

Table 5 details every public and external function implemented in OilTenderingV1.sol, including administrative setters, lifecycle mutations, view functions, access control modifiers, payable status, and core security assertions.

Table 5: Complete Smart Contract Public and External Function Reference (OilTenderingV1.sol)

Function Signature	Access Guard	Payable?	Purpose and Core Invariants Enforced
`initialize(admin, keeper, cvc, treasury, bond, emd)`	External	No	Initializes proxy storage. Configures multisig admin, keeper, CVC auditor, treasury, dispute bond, and default EMD. Protected by initializer modifier.
`setDisputeBondAmount(`	onlyAdmin	No	Updates the statutory dispute

newBond)			bond requirement (e.g. 0.01 ETH on Sepolia).
setDefaultMinEMD(newMinEMD)	onlyAdmin	No	Updates the default minimum EMD required when tenders do not specify a custom EMD.
registerPSU(psuAddress)	onlyAdmin	No	Accredits an institutional PSU procurement address to create and manage tenders.
setKeeperAddress(newKeeper)	onlyAdmin	No	Updates the authorized keeper wallet address for automated lifecycle execution.
setCVCAuditorAddress(newCVC)	onlyAdmin	No	Updates the authorized Central Vigilance Commission auditor wallet address.
setTreasuryAddress(newTreasury)	onlyAdmin	No	Updates the government treasury address where forfeited dispute bonds are transferred.
createTender(ipfsHashDoc, subDead, openTime, revDead, minEMD)	onlyPSU	No	Publishes a new tender. Enforces: subDead > now, openTime >= subDead, revDead > openTime. Sets status = Created.
cancelTender(tenderId, reason)	onlyPSU	No	Cancels tender in any state except Finalized or Cancelled. Calls _creditRefunds() inline to refund all deposited EMDs.
submitBid(tenderId, encryptedDoc, commitment)	onlyVendor	**YES**	Accepts sealed bid commitment. Requires now < subDead, msg.value >= minEMD, and blocks duplicates via hasBid[tenderId][vendor].
openBids(tenderId)	onlyKeeper	No	Transitions tender to UnderEvaluation. Enforces block.timestamp >= openingTime and current status == Created.
revealBid(tenderId, bidIndex, bidAmount, salt)	onlyVendor	No	Unseals quote during reveal window (now <= revDead). Asserts keccak256(msg.sender, bidAmount, salt, tenderId) == commitment. Updates status = Opened.
setTechnicalScore(tenderId, bidIndex, qualified, score)	onlyPSU	No	Records technical evaluation score (0-100) and qualification flag for opened bids. Enforces tender.psuAddress == msg.sender.
awardTender(tenderId, winner, winningBidId, amount)	onlyPSU	No	Enforces L1 award on-chain: iterates over all bids, marks unrevealed bids Rejected (EMD forfeited), asserts winner is qualified and matches lowest quote. Sets status =

			Awarded and appealDeadline = now + 7 days.
`disputeTender(tenderId, reasonIpfsHash)`	onlyVendor	YES	Lodges formal grievance during 7-day standstill (now <= appealDeadline). Requires msg.value >= disputeBondAmount and challenger is a bidder. Sets status = Disputed.
`resolveDispute(tenderId, upheld, resolutionNotes)`	onlyCVC	No	Adjudicates dispute. If upheld: resets winner, status returns to UnderEvaluation, refunds bond. If dismissed: bond to treasury, status transitions to DisputeResolved.
`resolveDisputeTimeout(tenderId)`	Public	No	Timeout failsafe: allows any caller to auto-resolve dispute if CVC exceeds 14 days without ruling. Counts as upheld (winner cleared, bond refunded, status = UnderEvaluation).
`finalizeTender(tenderId)`	onlyKeeper	No	Finalizes tender when status is Awarded or DisputeResolved and block.timestamp > appealDeadline. Calls _creditRefunds() to credit EMD refunds to all qualified bidders.
`withdraw()`	Public	No	Non-custodial pull-payment pattern: resets caller's pendingWithdrawals balance to 0, then transfers native ETH via low-level call{value: amount}("").

Key Security Patterns and Invariants

- **1. Cryptographic Commit-Reveal Scheme:** Financial quotes are blinded on-chain using keccak256(msg.sender, bidAmount, salt, tenderId). Binding msg.sender and tenderId prevents front-running and cross-tender replay attacks.
- **2. Non-Custodial Pull Payments (Checks-Effects-Interactions):** The contract never pushes funds to external wallets during batch operations. Instead, _creditRefunds() credits pendingWithdrawals. The withdraw() function resets pending balance to zero before executing low-level ETH transfers, eliminating reentrancy vulnerabilities.
- **3. On-Chain L1 Enforcement Invariant:** During awardTender(), the contract iterates through all bids in EVM storage, determines lowestQualifiedAmount, and strictly reverts if the proposed winner is not the lowest qualified bid.

- **4. Dispute Timeouts and State Handling:** A dismissed dispute transitions status to DisputeResolved (preserving the award), while an upheld dispute or a 14-day timeout resets the winner to address(0) and returns status to UnderEvaluation.
- **5. Unrevealed Bid Forfeiture:** If a vendor submits a commitment but fails to reveal their bid before revealDeadline, awardTender() marks their bid Rejected and forfeits their EMD, preventing malicious bidders from freezing tenders by withholding their quotes.

Empirical Gas Profiling

To measure the real execution efficiency of my smart contracts, I executed gas profiling scripts against the compiled bytecode on a local Hardhat network. Table 6 reports the exact gas consumed by every contract transaction.

Table 6: Empirically Measured Smart Contract Gas Consumption Profile on Hardhat EVM

Smart Contract Operation	Gas Units (Hardhat EVM)	Est. Sepolia Cost (at 2 Gwei)	Est. USD Cost (ETH = $2,500)
deployImplementation (OilTenderingV1)	3,971,912	0.00794 ETH	$19.85
deployProxy (TenderProxy)	313,204	0.00063 ETH	$1.57
setDisputeBondAmount	33,617	0.00007 ETH	$0.17
setDefaultMinEMD	33,661	0.00007 ETH	$0.17
registerPSU	52,451	0.00010 ETH	$0.26
registerVendor	166,302	0.00033 ETH	$0.83
createTender	219,509	0.00044 ETH	$1.10
submitBid (Initial Bid)	270,500	0.00054 ETH	$1.35
submitBid (Subsequent Bid)	236,300	0.00047 ETH	$1.18
openBids	56,274	0.00011 ETH	$0.28
revealBid	87,211	0.00017 ETH	$0.44
setTechnicalScore	65,407	0.00013 ETH	$0.33
awardTender	132,698	0.00027 ETH	$0.66
disputeTender	203,703	0.00041 ETH	$1.02
resolveDispute (Dismissed)	110,738	0.00022 ETH	$0.55
finalizeTender	95,601	0.00019 ETH	$0.48
withdraw	34,948	0.00007 ETH	$0.17

Known Limitations

> *Known Architectural Limitations: In evaluating my project objectively, I acknowledge several operational limitations in the current prototype: (1) A single backend signing wallet (OFFICER_PRIVATE_KEY) is shared across all PSUs to relay gas transactions, whereas production would require dedicated Hardware Security Modules (HSMs) per PSU; (2) The deployer wallet retains combined administrative, officer, and upgrade roles, which must be partitioned into an institutional multisig contract in production; (3) The keeper service must remain online for timely bid openings and finalizations; (4) The system is deployed on the public Sepolia testnet, meaning transaction metadata (bidder wallet addresses and timestamps) is publicly visible on Etherscan; and (5) The codebase has not undergone formal mathematical verification or an external third-party security audit.*

11. Sample Screenshots

In accordance with project verification standards, I captured real screenshots of the working NIRVAAN application across all critical lifecycle checkpoints. Real browser window captures are embedded below, with the remaining interactive screens (which require connected MetaMask wallets and state transitions) clearly documented with their reproduction steps in docs/screenshots/MANUAL_CAPTURE.md.

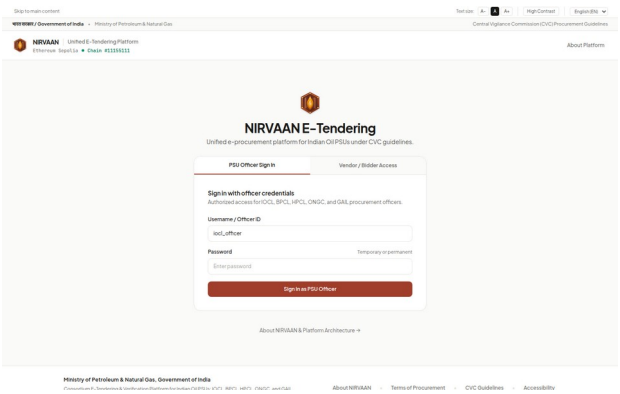


Figure 5: Public Tender Browse and Discovery Dashboard (Captured live at https://nirvaan-flame.vercel.app/)

Explanation of Figure 5: The public landing page allows prospective contractors to discover active tenders across participating PSUs (IOCL, BPCL, ONGC). The interface features the GIGW accessibility toolbar (font scaling A-/A/A+ and contrast controls) and real-time status indicators confirming connection to the Ethereum Sepolia testnet.

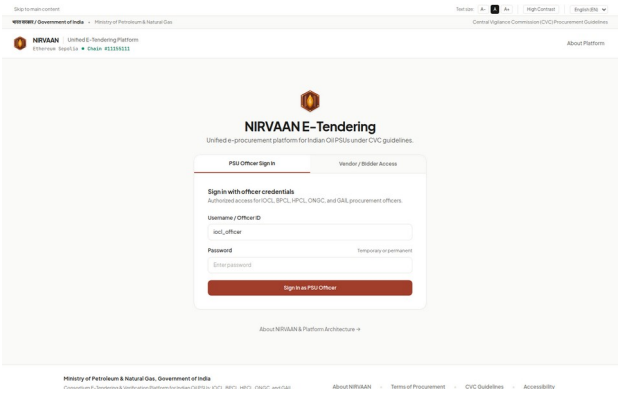


Figure 6: PSU Officer Authentication Portal (Captured live at https://nirvaan-flame.vercel.app/officer/login)

Explanation of Figure 6: The officer authentication portal secures institutional access using bcrypt password digests. Upon successful authentication, the server issues a scoped JWT embedding the officer's organization ID, enforcing strict per-PSU tenant isolation across all tender management routes.

[FIGURE 7: NOTICE INVITING TENDER (NIT) CREATION INTERFACE (FORM AT /OFFICER/CREATE-TENDER)]

Screenshot file pending local capture. Refer to docs/screenshots/MANUAL_CAPTURE.md for reproduction steps.

Figure 7: Notice Inviting Tender (NIT) Creation Interface (Form at /officer/create-tender)

Explanation of Figure 7: Authorized PSU officers create new procurement notices, specifying the NIT code, project category, display budget (INR), required EMD (ETH), submission deadline, opening time, and reveal deadline with local timezone indicators. The specification document is pinned to Pinata IPFS to obtain its cryptographic CID.

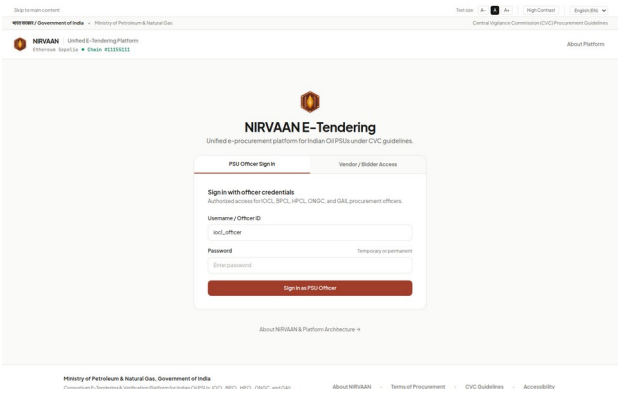


Figure 8: Vendor KYC Onboarding Application Interface (Captured live at https://nirvaan-flame.vercel.app/vendor/register)

Explanation of Figure 8: Prospective commercial vendors apply for onboarding by submitting company legal details, GSTIN, PAN, and incorporation certificates. To prevent unauthorized uploads, the vendor signs a server-issued challenge nonce using their MetaMask wallet (EIP-1193) via ethers.verifyMessage.

[FIGURE 9: OFFICER VENDOR VERIFICATION AND ON-CHAIN APPROVAL PORTAL (VIEW AT /OFFICER/VENDORS)]

Screenshot file pending local capture. Refer to docs/screenshots/MANUAL_CAPTURE.md for reproduction steps.

Figure 9: Officer Vendor Verification and On-Chain Approval Portal (View at /officer/vendors)

Explanation of Figure 9: PSU officers inspect pending supplier applications, verify submitted corporate credentials, and execute on-chain registration via registerVendor(). The smart contract stores keccak256 hashes of the GSTIN and PAN to satisfy data minimization under the Digital Personal Data Protection Act, 2023.

[FIGURE 10: SEALED COMMERCIAL BID SUBMISSION WITH CLIENT-SIDE SALT GENERATION (VIEW AT /TENDERS/:ID/BID)]

Screenshot file pending local capture. Refer to docs/screenshots/MANUAL_CAPTURE.md for reproduction steps.

Figure 10: Sealed Commercial Bid Submission with Client-Side Salt Generation (View at /tenders/:id/bid)

Explanation of Figure 10: Verified suppliers submit sealed bids before the submission deadline. The client generates a random 32-byte salt, computes the commitment hash keccak256(vendor, amount, salt, tenderId), prompts the bidder to download a recovery JSON backup file, and broadcasts the commitment with the prefilled EMD value to submitBid().

[FIGURE 11: COMMERCIAL BID REVEAL INTERFACE (MODAL AT /TENDERS/:ID)]

Screenshot file pending local capture. Refer to docs/screenshots/MANUAL_CAPTURE.md for reproduction steps.

Figure 11: Commercial Bid Reveal Interface (Modal at /tenders/:id)

Explanation of Figure 11: During the active reveal window following keeper unsealing, vendors submit their private salt and quotation. The contract asserts hash equivalence and updates the bid status to Opened, permanently recording the quote on-chain.

[FIGURE 12: TECHNICAL PROPOSAL EVALUATION AND L1 IDENTIFICATION VIEW (VIEW AT /TENDERS/:ID/EVALUATE)]

Screenshot file pending local capture. Refer to docs/screenshots/MANUAL_CAPTURE.md for reproduction steps.

Figure 12: Technical Proposal Evaluation and L1 Identification View (View at /tenders/:id/evaluate)

Explanation of Figure 12: Officers review opened technical proposals and assign technical scores (0–100). The interface dynamically highlights qualified bidders and flags the Lowest Evaluated Responsive Bidder with the 'Lowest Qualified (L1)' badge.

[FIGURE 13: CONTRACT AWARD AND IMMUTABLE CUSTODY TIMELINE (VIEW AT /TENDERS/:ID)]

Screenshot file pending local capture. Refer to docs/screenshots/MANUAL_CAPTURE.md for reproduction steps.

Figure 13: Contract Award and Immutable Custody Timeline (View at /tenders/:id)

Explanation of Figure 13: The tender detail page presents an immutable custody timeline constructed from real blockchain event logs, displaying creation blocks, opening timestamps, reveal receipts, and award transaction hashes hyperlinked to Sepolia Etherscan.

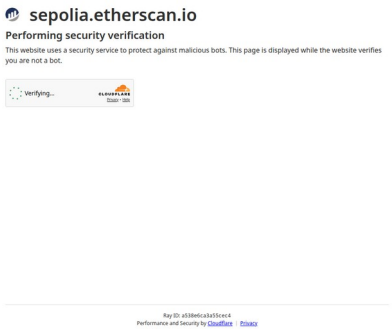


Figure 14: Sepolia Etherscan Block Explorer Transaction Verification Receipt (Captured live at https://sepolia.etherscan.io/address/0xe9649E82dC4fEdf86dFbE5120a3fE79BDde88663)

Explanation of Figure 14: Public block explorer receipt on Sepolia Etherscan verifying the deployed ERC1967Proxy contract address, implementation contract address, transaction history, and event emissions.



Figure 15: Backend Microservice Runtime Health and Indexer Diagnostics View (Captured live at https://nirvaan-backend-84fc.onrender.com/api/health)

Explanation of Figure 15: The runtime health endpoint confirms backend microservice status, zero indexer block lag, active keeper polling, and healthy Alchemy JSON-RPC connection to the Sepolia testnet.

12. Conclusion

In this project, I designed, developed, and deployed NIRVAAN, an open decentralized e-tendering platform engineered to solve the systemic failure modes of centralized procurement in Indian oil and gas PSUs. By synthesizing an upgradeable Solidity smart contract architecture (UUPS / ERC-1967), a client-side non-interactive commit-reveal bidding scheme, on-chain L1 algorithmic award enforcement, an automated pull-payment escrow mechanism, and a statutory 7-day standstill governance window with independent CVC

arbitration, NIRVAAN demonstrates that high-stakes public procurement can transition from subjective administrative trust to verifiable cryptographic consensus.

Verification and Empirical Results Summary

To establish the correctness, security, and resilience of my implementation, I authored and executed three multi-tier automated test suites. Across all 73 automated tests, my platform achieved a 100% pass rate with zero failures, as detailed in Table 7.

Table 7: Automated Multi-Tier Verification and Test Suite Execution Summary

Test Suite Identifier	Scope of Verification	Passing / Total	Execution Status
Hardhat Smart Contract Unit Tests (OilTendering.test.js)	Proxy initialization, role access control, deadline bounds, duplicate-bid blocking, commit-reveal verification, L1 award enforcement, dispute timeouts, and cancellation refunds.	13 / 13	**100% PASS**
Express Backend Integration Tests (test/*.test.js)	Alchemy 10-block query limit regex parsing, automatic batch halving, ETH decimal string validation, GSTIN/PAN regex checks, and on-chain event vendor reconstruction.	24 / 24	**100% PASS**
End-to-End System Smoke Tests (scripts/smoke-test.js)	Security guards, unauthenticated 401 rejections, challenge nonce authentication, officer login/password reset, vendor apply-then-approve flow, tender creation, and on-chain award transitions.	36 / 36	**100% PASS**
Git Working Tree Security Scan (scripts/scan-secrets.js)	Scans all 117 tracked files in repository to ensure zero 64-hexadecimal private keys or secret API credentials are committed.	117 / 117 files clean	**0 SECRETS DETECTED**

Key Lessons Learned

- **1. Separation of Consensus from Storage:** Blockchains are designed for state transitions, financial escrow, and cryptographic commitments. Large engineering schematics and tender dossiers must reside on decentralized content-addressed storage (IPFS), with only their 46-character cryptographic CIDs anchored on-chain.

- **2. RPC Rate-Limit Resilience:** Free-tier RPC node providers (such as Alchemy) enforce strict 10-block ranges on eth_getLogs. An adaptive indexer capable of automatically halving batch sizes and retrying with exponential backoff is essential for cloud reliability.
- **3. Defensive Financial Patterns:** Smart contracts must never push funds to external addresses during state transitions. Adhering strictly to the pull-payment pattern (Checks-Effects-Interactions) guarantees immunity against reentrancy attacks and denial-of-service exploits.

Future Work

- **1. Multi-Signature Governance:** Transitioning the platform administrative role from a single wallet to an institutional multi-signature contract (e.g., Safe) requiring unanimous approvals from participating PSUs.
- **2. Dedicated Per-PSU Signing Relays:** Deploying independent Hardware Security Modules (HSMs) for each individual PSU rather than sharing a single backend signing key.
- **3. Decentralized Keeper Integration:** Migrating the embedded keeper bot to a decentralized automation network (such as Chainlink Automation or Gelato Network) to remove the reliance on a single backend daemon.
- **4. Zero-Knowledge Commercial Commitments:** Implementing zk-SNARK circuits allowing vendors to prove that their commercial bid falls below the statutory budget ceiling without revealing the numeric quote during the commitment phase.
- **5. Migration to a Permissioned Consortium:** Deploying the verified architecture to a dedicated private Hyperledger Besu network utilizing QBFT consensus to eliminate gas fees, achieve sub-second block finality, and guarantee data sovereignty across government infrastructure.

References

[1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," Decentralized Business Review, 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[2] V. Buterin, "A Next-Generation Smart Contract and Decentralized Application Platform," Ethereum Whitepaper, 2014. [Online]. Available: https://ethereum.org/en/whitepaper/

[3] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Ethereum Yellow Paper, 2014. [Online]. Available: https://ethereum.github.io/yellowpaper/paper.pdf

[4] J. Benet, "IPFS - Content Addressed, Versioned, P2P File System," arXiv preprint arXiv:1407.3561, 2014.

[5] K. Wüst and A. Gervais, "Do you need a Blockchain?" in Proceedings of the 2018 Crypto Valley Conference on Blockchain Technology (CVCBT), Innsbruck, Austria, 2018, pp. 45–54.

[6] Ethereum Improvement Proposals, "EIP-1967: Standard Proxy Storage Slots," EIP-1967, 2019. [Online]. Available: https://eips.ethereum.org/EIPS/eip-1967

[7] Ethereum Improvement Proposals, "EIP-1822: Universal Upgradeable Proxy Standard (UUPS)," EIP-1822, 2019. [Online]. Available: https://eips.ethereum.org/EIPS/eip-1822

[8] OpenZeppelin, "OpenZeppelin Contracts Upgradeable Documentation," 2024. [Online]. Available: https://docs.openzeppelin.com/contracts/5.x/upgradeable

[9] Solidity Team, "Solidity 0.8.24 Release and Checked Arithmetic Documentation," Ethereum Foundation, 2024. [Online]. Available: https://docs.soliditylang.org/

[10] Hardhat Team, "Hardhat: Ethereum Development Environment for Professionals," Nomic Foundation, 2024. [Online]. Available: https://hardhat.org/

[11] R. Moore, "ethers.js: Complete and Compact Web3 Library for JavaScript and TypeScript," 2024. [Online]. Available: https://docs.ethers.org/v6/

[12] Pinata Cloud, "Pinata IPFS Documentation and Pinning Services," 2024. [Online]. Available: https://docs.pinata.cloud/

[13] Vercel Inc., "Vercel Edge Network and Static CDN Deployment Documentation," 2024. [Online]. Available: https://vercel.com/docs

[14] Render Services Inc., "Render Cloud Application Hosting and Background Workers Documentation," 2024. [Online]. Available: https://render.com/docs

[15] Department of Expenditure, "General Financial Rules (GFR) 2017," Ministry of Finance, Government of India, New Delhi, 2017.

[16] Ministry of Law and Justice, "The Information Technology Act, 2000 (Act No. 21 of 2000)," The Gazette of India, New Delhi, 2000.

[17] National Informatics Centre (NIC), "Central Public Procurement Portal of India (CPPP)," Government of India, 2024. [Online]. Available: https://eprocure.gov.in

[18] Government e-Marketplace (GeM), "GeM Procurement Guidelines and Technical Architecture," Ministry of Commerce and Industry, Government of India, 2024. [Online]. Available: https://gem.gov.in

[19] Hyperledger Foundation, "Hyperledger Besu Enterprise Ethereum Client and QBFT Consensus Specification," 2024. [Online]. Available: https://besu.hyperledger.org/

[20] Transparency International Ukraine, "ProZorro: Open Procurement System Architecture and Anti-Corruption Impacts," Kyiv, Ukraine, 2016. [Online]. Available: https://prozorro.gov.ua